```
# Implementasi Red-Black Tree
class Node:
    def __init__(self, key):
        self.key = key
        self.color = True  # True = RED, False = BLACK
        self.left = None
        self.right = None
        self.parent = None

class RedBlackTree:
    def __init__(self):
        # Node NIL (Sentinel)
        self.NIL = Node(0)
        self.NIL.color = False  # Black
        self.NIL.left = None
        self.NIL.right = None
        self.root = self.NIL

    def left_rotate(self, x):
        y = x.right
        x.right = y.left

        if y.left != self.NIL:
            y.left.parent = x

        y.parent = x.parent

        if x.parent is None:
            self.root = y
        elif x == x.parent.left:
            x.parent.left = y
        else:
            x.parent.right = y

        y.left = x
        x.parent = y

    def right_rotate(self, x):
        y = x.left
        x.left = y.right

        if y.right != self.NIL:
            y.right.parent = x

        y.parent = x.parent

        if x.parent is None:
            self.root = y
        elif x == x.parent.right:
            x.parent.right = y
        else:
            x.parent.left = y
```

```
        y.right = x
        x.parent = y

    def insert(self, key):
        z = Node(key)
        z.parent = None
        z.key = key
        z.left = self.NIL
        z.right = self.NIL
        z.color = True  # New node is RED

        y = None
        x = self.root

        while x != self.NIL:
            y = x
            if z.key < x.key:
                x = x.left
            else:
                x = x.right

        z.parent = y
        if y is None:
            self.root = z
        elif z.key < y.key:
            y.left = z
        else:
            y.right = z

        # Jika new node adalah root
        if z.parent is None:
            z.color = False
            return

        # Jika kakek None, langsung return
        if z.parent.parent is None:
            return

        self.insert_fixup(z)

    def insert_fixup(self, z):
        while z.parent and z.parent.color == True:
            if z.parent == z.parent.parent.left:
                uncle = z.parent.parent.right
                if uncle.color == True:  # Case 1: Uncle Red
                    z.parent.color = False
                    uncle.color = False
                    z.parent.parent.color = True
                    z = z.parent.parent
                else:
                    if z == z.parent.right:  # Case 2: Triangle
                        z = z.parent
                        self.left_rotate(z)
                    # Case 3: Line
                    z.parent.color = False
```

```
                    z.parent.parent.color = True
                    self.right_rotate(z.parent.parent)
            else:  # Symmetric
                uncle = z.parent.parent.left
                if uncle.color == True:
                    z.parent.color = False
                    uncle.color = False
                    z.parent.parent.color = True
                    z = z.parent.parent
                else:
                    if z == z.parent.left:
                        z = z.parent
                        self.right_rotate(z)
                    z.parent.color = False
                    z.parent.parent.color = True
                    self.left_rotate(z.parent.parent)
            if z == self.root:
                break
        self.root.color = False

    # Utility function untuk print tree
    def print_tree(self, node, indent, last):
        if node != self.NIL:
            print(indent, end='')
            if last:
                print("R----", end='')
                indent += "   "
            else:
                print("L----", end='')
                indent += "|  "

            color = "MERAH" if node.color else "HITAM"
            print(f"{node.key} ({color})")
            self.print_tree(node.left, indent, False)
            self.print_tree(node.right, indent, True)

# ======================
# Contoh Penggunaan
# ======================
if __name__ == "__main__":
    rbt = RedBlackTree()
    data = [10, 20, 30, 15, 25, 5, 3]

    for num in data:
        rbt.insert(num)

    print("Struktur Red-Black Tree:")
    rbt.print_tree(rbt.root, "", True)
```

```
Struktur Red-Black Tree:
R----20 (HITAM)
   L----10 (MERAH)
   |  L----5 (HITAM)
   |  |  L----3 (MERAH)
   |  R----15 (HITAM)
```

```
   R----30 (HITAM)
      L----25 (MERAH)
```

```
RED = True
BLACK = False

# The structure for Node is typically defined as a class in Python.
# A 'Node' class with similar attributes is already defined in another cell.
# For example:
# class Node:
#     key: int
#     color: bool
#     left: 'Node'
#     right: 'Node'
#     parent: 'Node'
```

```
def LeftRotate(tree_instance, x):
    y = x.right
    x.right = y.left

    if y.left != tree_instance.NIL:
        y.left.parent = x

    y.parent = x.parent

    if x.parent is None:
        tree_instance.root = y
    elif x == x.parent.left:
        x.parent.left = y
    else:
        x.parent.right = y

    y.left = x
    x.parent = y

def RightRotate(tree_instance, x):
    y = x.left
    x.left = y.right

    if y.right != tree_instance.NIL:
        y.right.parent = x

    y.parent = x.parent

    if x.parent is None:
        tree_instance.root = y
    elif x == x.parent.right:
        x.parent.right = y
    else:
        x.parent.left = y

    y.right = x
    x.parent = y
```

```
def Insert(tree_instance, z):
    y = None
    x = tree_instance.root

    while x != tree_instance.NIL:
        y = x
        if z.key < x.key:
            x = x.left
        else:
            x = x.right

    z.parent = y
    if y is None:
        tree_instance.root = z
    elif z.key < y.key:
        y.left = z
    else:
        y.right = z

    z.left = tree_instance.NIL
    z.right = tree_instance.NIL
    z.color = RED

    InsertFixup(tree_instance, z)

def InsertFixup(tree_instance, z):
    while z.parent is not None and z.parent.color == RED:
        # Ensure z.parent.parent exists before accessing it
        if z.parent.parent is None:
            break

        if z.parent == z.parent.parent.left:
            uncle = z.parent.parent.right

            # Kasus 1: Paman Merah
            if uncle.color == RED:
                z.parent.color = BLACK
                uncle.color = BLACK
                z.parent.parent.color = RED
                z = z.parent.parent
            else:
                # Kasus 2: Segitiga
                if z == z.parent.right:
                    z = z.parent
                    LeftRotate(tree_instance, z)

                # Kasus 3: Garis
                z.parent.color = BLACK
                z.parent.parent.color = RED
                RightRotate(tree_instance, z.parent.parent)
        else:
            # Simetris (kanan/kiri)
            uncle = z.parent.parent.left

            if uncle.color == RED:
```

```
                if uncle.color == RED:
                    z.parent.color = BLACK
                    uncle.color = BLACK
                    z.parent.parent.color = RED
                    z = z.parent.parent
                else:
                    if z == z.parent.left:
                        z = z.parent
                        RightRotate(tree_instance, z)
                    z.parent.color = BLACK
                    z.parent.parent.color = RED
                    LeftRotate(tree_instance, z.parent.parent)

            if z == tree_instance.root:
                break
        tree_instance.root.color = BLACK
```